



Projet de Recherche Opérationnelle

Flot maximum et flot maximum de coût minimum

Bence Di Placido

May 8, 2026

Contents

1	Introduction	2
2	Format d'entrée et sortie du programme	2
3	Structures de données	2
3.1	Représentation des noeuds	2
3.2	Représentation des arcs	3
3.3	Représentation du graphe	3
3.4	Graphe résiduel	3
4	Construction du graphe	4
5	Algorithme de flot maximum	4
6	Algorithme de flot maximum de coût minimum	4
7	Visualisation	5
8	Exemple de résolution en coût minimum	6
9	Tests	8
10	Conclusion	8

1 Introduction

Le but de ce projet est de manipuler des graphes de flot en Java. On part d'un graphe orienté où chaque arc possède une capacité, et éventuellement un coût. Le programme doit lire ce graphe depuis un fichier texte, calculer une solution, puis produire une visualisation du graphe avant et après le calcul en fournissant le flot maximum, la min cut et le coût si demandé.

Deux problèmes sont traités. Le premier est le problème classique du flot maximum : on cherche à envoyer le plus de flot possible depuis une source **s** vers un puits **t**, sans dépasser les capacités des arcs. Le deuxième est une variante plus complète : parmi les flots maximums possibles, on veut trouver un flot dont le coût total est minimum.

Le projet a donc été organisé autour de trois étapes assez simples :

1. construire le graphe à partir d'un fichier d'entrée ;
2. appliquer l'algorithme choisi ;
3. exporter le résultat en DOT/PDF pour vérifier visuellement le flot, les coûts et la coupe minimale.

J'ai essayé de garder une structure lisible plutôt que de tout mettre dans une seule classe. Même si le projet reste de taille raisonnable, séparer les modèles, les constructeurs et les algorithmes permet de mieux comprendre où chaque responsabilité se trouve.

2 Format d'entrée et sortie du programme

Le fichier d'entrée contient d'abord une ligne avec le nombre de noeuds, le nombre d'arcs, l'indice de la source et l'indice du puits :

```
nombre_noeuds nombre_arcs source puits
```

Ensuite, chaque ligne décrit un arc :

```
source destination capacite cout
```

Par exemple, le graphe utilisé dans les tests contient 6 noeuds, 10 arcs, la source d'indice 5 et le puits d'indice 4. Dans le programme, la source est renommée **s** et le puits **t**, ce qui rend la lecture des graphes plus claire.

Après l'exécution, le programme génère deux versions du graphe :

- une version initiale, avant le calcul ;
- une version résultat, avec les flots calculés, le flot maximum, le coût total et les arcs de la coupe minimale mis en évidence.

3 Structures de données

3.1 Représentation des noeuds

Les noeuds sont représentés par une classe abstraite **Node**. Elle contient un identifiant, un nom et une liste d'arcs sortants. Trois classes spécialisées héritent ensuite de **Node** :

- **StartNode** pour la source ;
- **MiddleNode** pour les noeuds intermédiaires ;
- **EndNode** pour le puits.

Ce choix n'est pas obligatoire pour résoudre le problème : on aurait pu utiliser une seule classe `Node` avec un attribut indiquant le type du noeud. J'ai préféré les classes séparées car elles rendent le modèle plus explicite. Quand on lit le code, on voit directement quel noeud joue le rôle de source et quel noeud joue le rôle de puits.

3.2 Représentation des arcs

Chaque arc est représenté par la classe `Arc`. Elle stocke :

- la source et la destination ;
- la capacité restante ;
- la capacité initiale ;
- le flot actuellement envoyé ;
- le coût de passage sur l'arc.

La capacité initiale est gardée séparément de la capacité courante. Ce choix est utile car les algorithmes modifient les capacités dans le graphe résiduel, alors que l'affichage final doit encore connaître la capacité de départ. Sans cette information, il serait plus difficile d'afficher des labels du type `flot/capacité`.

3.3 Représentation du graphe

La classe `Graph` regroupe la source, le puits, les noeuds intermédiaires et plusieurs informations de résultat : le flot maximum, le coût total et la coupe minimale. Le graphe est représenté principalement avec des listes d'adjacence : chaque noeud possède sa liste d'arcs sortants.

J'ai choisi les listes d'adjacence plutôt qu'une matrice de capacités. Une matrice est pratique pour accéder très vite à une capacité entre deux noeuds, mais elle oblige à réserver une case pour chaque paire de noeuds, même quand il n'y a pas d'arc. Ici les graphes sont naturellement orientés et peuvent être plutôt creux, donc une liste d'adjacence est plus simple et plus économique en mémoire. Elle correspond aussi bien aux parcours utilisés par BFS et Bellman-Ford, qui inspectent les arcs sortants d'un noeud.

3.4 Graphe résiduel

Le graphe résiduel est construit par `ResidualGraphBuilder`. Pour chaque arc original, le constructeur crée deux arcs :

- un arc direct, avec la capacité disponible ;
- un arc inverse, avec une capacité initialement nulle et un coût opposé.

L'arc inverse est essentiel. Il permet à l'algorithme de revenir sur une décision ancienne si un meilleur chemin est trouvé ensuite. Dans le cas du coût minimum, son coût négatif représente le fait qu'annuler du flot annule aussi le coût payé auparavant.

Le graphe garde aussi plusieurs `Map` :

- une correspondance entre arc résiduel et arc original ;
- une indication pour savoir si un arc résiduel est inverse ou non ;
- une correspondance entre chaque arc résiduel et son arc opposé.

Ces tables ajoutent un peu de complexité, mais elles évitent de modifier directement le graphe original pendant la recherche. Le résultat final reste ainsi plus propre : le graphe original porte les flots finaux, tandis que le graphe résiduel sert au calcul.

Une autre possibilité aurait été de stocker directement une référence vers l'arc original dans chaque arc résiduel. J'ai préféré utiliser des `Map`, car cela garde la classe `Arc` assez générale : un arc ne sait pas s'il vient du graphe original ou du graphe résiduel. Les correspondances sont gérées par le constructeur du graphe résiduel, ce qui centralise cette logique au même endroit.

4 Construction du graphe

La classe `GraphBuilder` lit le fichier texte ligne par ligne. Les lignes vides et les commentaires sont ignorés, ce qui rend les fichiers d'entrée plus faciles à écrire et à modifier.

La première ligne permet de créer tous les noeuds. Ensuite, chaque ligne d'arc est vérifiée puis convertie en objet `Arc`. Le programme refuse les capacités négatives, les arcs vers des noeuds inexistants et les fichiers dont le nombre d'arcs ne correspond pas à l'en-tête.

J'ai choisi de valider les erreurs dès la lecture plutôt que de laisser l'algorithme tomber plus tard sur un graphe incohérent.

5 Algorithme de flot maximum

Pour le flot maximum sans coût, le projet utilise Ford-Fulkerson dans sa version Edmonds-Karp. Le principe est le suivant :

1. construire le graphe résiduel ;
2. chercher un chemin augmentant de s vers t ;
3. envoyer sur ce chemin le maximum possible, c'est-à-dire le minimum des capacités restantes du chemin ;
4. mettre à jour les arcs directs et inverses du graphe résiduel ;
5. recommencer tant qu'il existe un chemin augmentant.

La différence avec un Ford-Fulkerson très général est que le chemin augmentant est trouvé avec un BFS. Cela correspond à Edmonds-Karp. Ce choix est plus stable qu'une recherche arbitraire en profondeur, car le BFS prend toujours un chemin avec un nombre minimal d'arcs. En pratique, cela évite certains comportements où Ford-Fulkerson peut faire beaucoup de petites augmentations peu efficaces.

Après la fin de l'algorithme, les noeuds encore atteignables depuis la source dans le graphe résiduel sont calculés. Les arcs originaux qui partent d'un noeud atteignable vers un noeud non atteignable forment une coupe minimale. Le programme les garde pour les afficher en rouge dans le fichier DOT.

6 Algorithme de flot maximum de coût minimum

Pour la version avec coûts, l'idée reste proche : on augmente le flot dans le graphe résiduel. La différence est le choix du chemin. Au lieu de prendre le plus court chemin en nombre d'arcs, on prend le chemin de coût total minimal.

Pour trouver ce chemin, le projet utilise Bellman-Ford. Ce choix est important parce que le graphe résiduel contient des arcs inverses avec des coûts négatifs. Dijkstra aurait été plus rapide dans le cas de coûts tous positifs, mais il ne fonctionne pas directement avec des coûts négatifs. Bellman-Ford est donc plus adapté ici, même s'il est moins rapide.

À chaque itération :

1. Bellman-Ford cherche un chemin de coût minimal entre **s** et **t** ;
2. on calcule la capacité limitante du chemin ;
3. on augmente le flot de cette quantité ;
4. on ajoute au coût total le coût du chemin multiplié par la quantité de flot envoyée.

Le programme vérifie aussi la présence d'un cycle de coût négatif atteignable. Ce contrôle est nécessaire : avec un tel cycle, on pourrait diminuer le coût indéfiniment dans le graphe résiduel, ce qui rendrait la notion de coût minimum problématique.

7 Visualisation

L'export est fait avec Graphviz. La classe **DotPdfExporter** écrit d'abord un fichier DOT, puis appelle la commande **dot** pour produire un PDF. Les arcs affichent le flot et la capacité en bleu, puis le coût en rouge. La source est colorée en vert et le puits en bleu.

Cette visualisation est utile pour vérifier rapidement que le résultat a du sens. Par exemple, sur le graphe fourni, les tests indiquent un flot maximum de 15. Avec l'algorithme de coût minimum, le coût total attendu est 149.

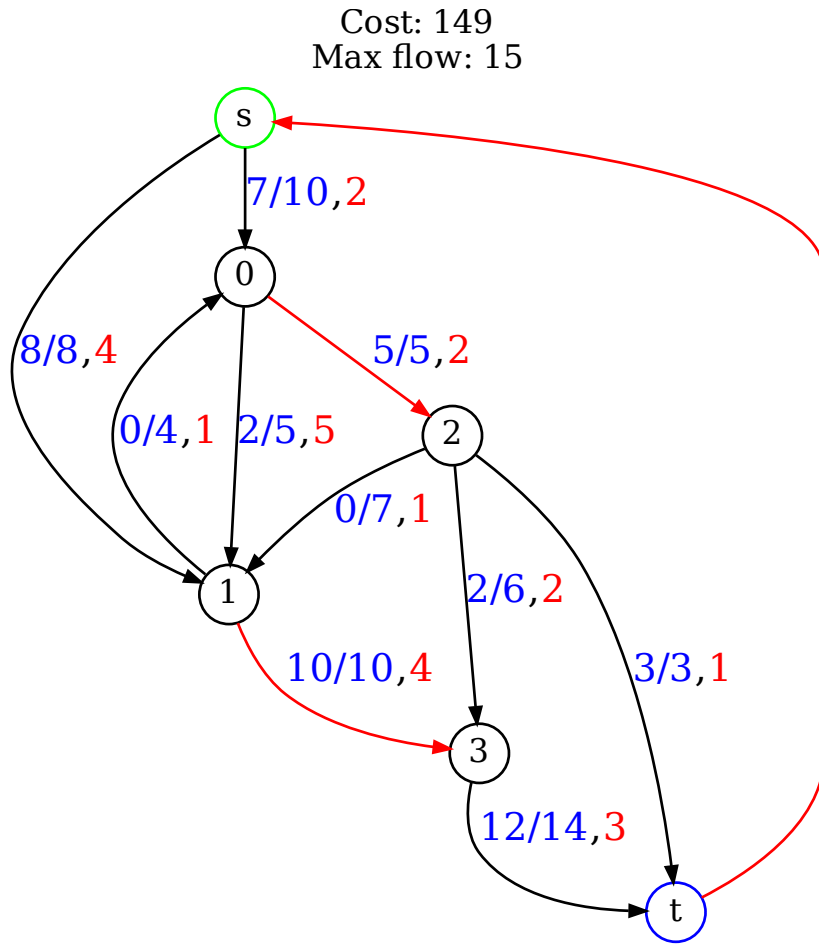


Figure 1: Graphe obtenu après le calcul sur l'exemple fourni.

8 Exemple de résolution en coût minimum

Pour illustrer le fonctionnement de l'algorithme de flot maximum de coût minimum, on peut regarder le graphe fourni dans les ressources. Au départ, aucun flot n'a encore été envoyé. Les arcs sont seulement définis par leur capacité et leur coût. Dans les figures, le label bleu correspond à $\text{flot}/\text{capacité}$, et le nombre rouge correspond au coût de l'arc.

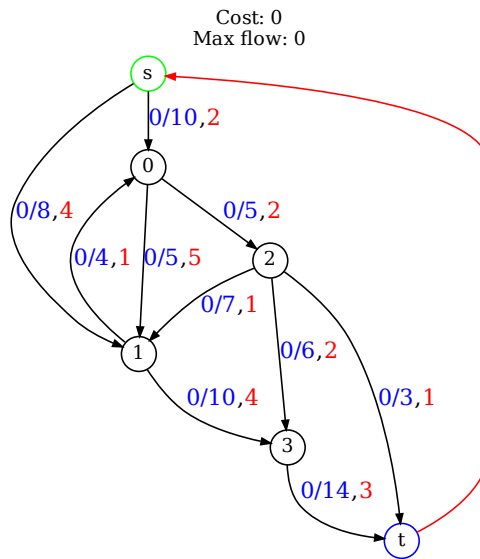


Figure 2: Graphe initial avant le calcul : toutes les valeurs de flot sont à 0.

Sur ce graphe initial, il faut trouver la solution avec le flot max qui coûte le moins cher. C'est pour cela que l'algorithme ne prend pas simplement le premier chemin disponible : à chaque étape, il cherche un chemin augmentant de coût minimal dans le graphe résiduel.

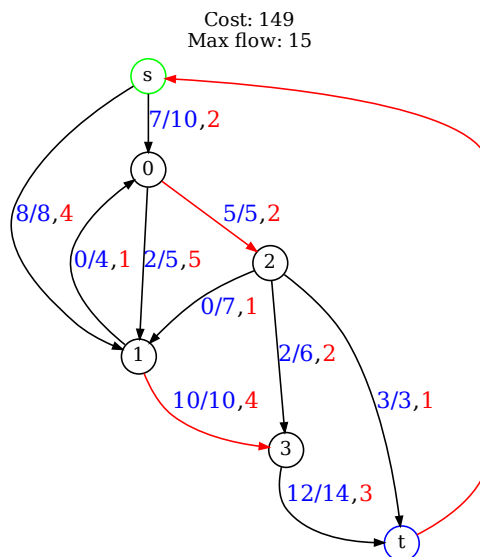


Figure 3: Graphe après résolution avec l'algorithme min-cost max-flow.

Le résultat obtenu est un flot maximum de 15 pour un coût total de 149. On voit que les deux arcs qui sortent de la source sont utilisés : $s \rightarrow 0$ transporte 7 unités sur 10 possibles, et $s \rightarrow 1$ transporte 8 unités sur 8. L'arc $s \rightarrow 1$ est donc saturé.

Du côté du puits, le flot arrive par deux arcs : $2 \rightarrow t$ transporte 3 unités sur 3, et $3 \rightarrow t$

transporte 12 unités sur 14. Le total entrant dans t est donc bien $3 + 12 = 15$, ce qui correspond au flot maximum affiché.

Les arcs en rouge représentent la coupe minimale trouvée après le calcul. Ici, on retrouve notamment $0 \rightarrow 2$ et $1 \rightarrow 3$, qui sont saturés dans la solution. Cela signifie qu'une fois ces passages pleins, il n'existe plus de chemin résiduel permettant d'envoyer davantage de flot de s vers t . Le programme ne se contente donc pas de donner une valeur numérique : il permet aussi de voir quels arcs limitent le réseau.

Le coût total de 149 vient de la somme des flots envoyés sur chaque arc, multipliés par leurs coûts. Par exemple, envoyer du flot sur un arc de coût 1 est préférable à capacité égale, mais l'algorithme doit aussi respecter les capacités et parfois utiliser des arcs plus chers pour atteindre le flot maximum. C'est exactement l'intérêt du min-cost max-flow : garder le débit maximal, tout en choisissant la combinaison de chemins la moins coûteuse possible.

9 Tests

Des tests JUnit sont présents pour vérifier les deux algorithmes. Pour Ford-Fulkerson, le test vérifie que le flot maximum du graphe donné en exemple vaut 15 et que l'algorithme peut être relancé deux fois sur le même graphe sans conserver un état incorrect.

Pour le flot de coût minimum, le test vérifie que le flot maximum vaut aussi 15 et que le coût total vaut 149 pour le graphe donné en exemple. Un autre test construit un petit graphe avec un cycle de coût négatif atteignable et vérifie que l'algorithme le refuse.

Ces tests ne prouvent pas que le programme est parfait, mais ils couvrent les cas principaux du projet : calcul du flot, calcul du coût et détection d'une situation problématique.

10 Conclusion

Pour conclure, ce projet montre surtout que les algorithmes de flot ne servent pas seulement à résoudre des graphes abstraits. Edmonds-Karp permet de trouver la quantité maximale que l'on peut faire passer d'une source vers un puits, par exemple dans un réseau de transport, de routes, de données ou de production. La version min-cost max-flow ajoute une idée importante : on garde le meilleur débit possible, mais en cherchant en plus à réduire le coût total.

En modélisant des problèmes de tous les jours sous forme de graphe, on peut donc obtenir des réponses assez concrètes : combien peut-on envoyer au maximum, quels passages bloquent le réseau, ou quelle organisation donne le même résultat pour un coût plus faible. C'est ce qui rend ces algorithmes intéressants. Une fois le problème bien traduit en source, puits, capacités et coûts, le calcul permet de faire apparaître des solutions qui ne sont pas toujours évidentes à trouver.